Departamento de Ciencias de la Computación (DCCO)

Desarrollo Software Seguro

Proyecto Integrador del Primer Parcial

Backend V1: autenticación JWT/RBAC en NestJS y detección de esteganografía LSB en imágenes

Integrantes

Dennison Chalcán
Fernando Sandoval
Jerson Llumiquinga

Fecha de entrega

04 de noviembre de 2026

Documento técnico del sistema implementado. Versión 2.0, adaptada a la implementación real en NestJS.
Repositorio: `secure-chat-proyecto-DJF`

Índice

1. Resumen **1**

2. Introducción **1**

2.1. Problema y contexto 1

2.2. Objetivo general 2

2.3. Objetivos específicos 2

2.4. Alcance y limitaciones 2

3. Arquitectura de la solución **2**

3.1. Vista de componentes 2

3.2. Flujo de una solicitud de análisis 4

3.3. Límites de confianza 4

3.4. Tecnologías 5

4. Modelo de identidad y datos **5**

4.1. Modelo entidad–relación 5

4.2. Esquema principal 5

4.3. Datos semilla 7

4.4. Persistencia de STEGO 8

5. Microservicio AUTH **8**

5.1. Contraseñas e identidad 8

5.2. JWT RS256 8

5.3. RBAC vigente 9

5.4. Autorización interna entre servicios 9

5.5. Revocación y cierre de sesión 9

5.6. Endpoints de AUTH 10

5.7. Controles adicionales 10

6. Microservicio STEGO **11**

6.1. Pipeline de validación 11

6.2. Detección de esteganografía LSB 11

6.3. Clasificación y política 12

6.4. Resultados, aislamiento y trazabilidad 13

6.5. Endpoints de STEGO 13
6.6. Ejemplo de resultado 13

7. Seguridad, riesgos y controles **14**
7.1. Enfoque de seguridad 14
7.2. Decisiones de diseño relevantes 16

8. Pruebas y resultados **17**
8.1. Estrategia 17
8.2. Pruebas unitarias 17
8.3. Pruebas de extremo a extremo de AUTH 17
8.4. Cobertura de los escenarios T01–T12 17
8.5. Resultados sobre imágenes sintéticas 18
8.6. Reproducibilidad 19

9. Anexos **20**
9.1. Link del Repositorio 20
9.2. Funcionalidad del aplicativo 21
9.3. Estructura del entregable 22
9.4. Resumen de contratos OpenAPI 22

10. Conclusiones **23**

Referencias **23**

Índice de figuras

1. Arquitectura de componentes y límites de confianza. Las bases de datos no publican puertos al host; las APIs se publican solo en `127.0.0.1`. 3
2. Modelo entidad–relación de AUTH. 6
3. Acceso válido al sistema usando las credenciales de ADMIN 21
4. Validación de esteganografía de un archivo PDF (Rechaza el archivo) 21
5. Aceptación de un archivo permitido (png) 21
6. Intento de analizar esteganografía desde rol usuario (Insuficientes permisos) . . . 22

Índice de tablas

1. Servicios de Backend V1. 3
2. Controles en los límites de confianza. 4
3. Tecnologías y propósito. 5
4. Usuarios semilla y roles. 7
5. Claims del JWT. 8
6. Matriz de roles y permisos. 9
7. Endpoints de AUTH. 10
8. Validaciones de entrada de STEGO. 11
9. Política de clasificación. 12
10. Endpoints de STEGO. 13
11. Matriz de activos, amenazas y controles esperados. 15
12. Resultados de pruebas unitarias ejecutadas. 17
13. Cobertura de los escenarios T01–T12. 18
14. Fixtures sintéticos y comportamiento verificado. 19

1. Resumen

El presente proyecto implementa **Backend V1**, la primera entrega del backend de una aplicación de salas de chat seguras, compuesta por dos microservicios REST independientes desarrollados con **NestJS**. El primero, **AUTH** (`ms-auth-nestjs`), administra identidades, roles, asignaciones usuario–rol, emisión y revocación de tokens JWT firmados con RSA (RS256), y autorización interna entre servicios. El segundo, **STEGO** (`ms-stego-nestjs`), recibe imágenes PNG y BMP, valida su estructura con *magic bytes*, ejecuta un detector de esteganografía LSB basado en entropía, igualación de pares y chi-cuadrado, y clasifica el archivo como limpio, no concluyente o sospechoso.

La solución aplica los principios *Secure by Design*, *Zero Trust* y *Shift Left*: el rol efectivo se consulta en PostgreSQL en cada operación, el token no se considera una verdad permanente, los archivos se validan antes de procesarse y la comunicación entre servicios se autentica con una clave de servicio de 256 bits mediante el encabezado `X-Service-Key`. La clave privada RSA y las bases de datos no se exponen fuera de la red interna de Docker.

El sistema se ejecuta con Docker Compose en seis servicios autónomos, incluye migraciones y datos semilla, documentación OpenAPI/Swagger, y una suite de pruebas automatizadas. La ejecución local reportó **83/83 pruebas unitarias aprobadas** (36 de AUTH y 47 de STEGO). La suite e2e de AUTH comprende **30 pruebas** contra PostgreSQL real y se ejecuta con Docker en marcha.

Resultado verificable

Al cierre de esta adaptación, `npm test` reportó **36/36 pruebas aprobadas** en AUTH y **47/47** en STEGO. Los servicios quedan publicados únicamente en loopback: AUTH en `http://localhost:3001` y STEGO en `http://localhost:3002`.

Palabras clave: microservicios; NestJS; JWT; RBAC; Argon2id; esteganografía; desarrollo seguro.

2. Introducción

2.1. Problema y contexto

Los sistemas que reciben archivos de usuarios enfrentan dos riesgos simultáneos. Por un lado, deben garantizar que solo identidades autenticadas y autorizadas ejecuten operaciones sensibles. Por otro, deben impedir que archivos malformados, excesivos o manipulados provoquen denegación de servicio, ejecución de código o fuga de información. Cuando la autorización se resuelve únicamente en el cliente, o cuando el token se emite sin posibilidad de revocación, el control de acceso deja de ser vigente.

El proyecto aborda este problema mediante una separación estricta de responsabilidades: AUTH decide *quién es* el usuario y *qué puede hacer*; STEGO decide *si el archivo es procesable* y aplica la autorización recibida de AUTH antes de analizarlo. La separación permite razonar sobre

límites de confianza, auditar cada decisión y escalar o aislar cada servicio de forma independiente, con su propia base de datos y su propio ciclo de migraciones.

2.2. Objetivo general

Implementar un backend seguro de dos microservicios REST en NestJS que integre autenticación JWT con RBAC dinámico, comunicación autenticada entre servicios y análisis de esteganografía LSB en imágenes, con controles de validación, trazabilidad y pruebas reproducibles.

2.3. Objetivos específicos

- Diseñar un modelo de identidad con usuarios, roles y relación muchos a muchos.
- Emitir y verificar JWT RS256, con revocación por `tokenVersion` y consulta del rol efectivo en cada operación.
- Proteger los endpoints de STEGO mediante autorización interna de AUTH y vínculo de propiedad del resultado.
- Validar imágenes por extensión, *magic bytes*, tamaño, dimensiones y decodificación antes del análisis.
- Detectar indicios de esteganografía LSB y clasificar el archivo con una política explícita.
- Documentar la arquitectura, el contrato OpenAPI, los riesgos y la evidencia de pruebas.

2.4. Alcance y limitaciones

La versión implementa el backend y su operación local con Docker Compose. No incluye interfaz gráfica ni despliegue en nube. El detector es una herramienta de triage: reduce falsos positivos frente a una simple suma de LSB, pero no demuestra criptográficamente la ausencia de información oculta. Los formatos soportados son PNG y BMP verificados por extensión y *magic bytes*; el contenido se normaliza con `sharp` a sRGB sin canal alfa. Los archivos de más de 10 MiB, con más de 10000 píxeles por lado o con más de 40 megapíxeles se rechazan de forma segura. STEGO no conserva el archivo original ni una copia física de cuarentena: la acción `cuarentena` es una decisión lógica registrada en la base de datos.

3. Arquitectura de la solución

3.1. Vista de componentes

La Tabla 1 resume los servicios y su responsabilidad. AUTH es el único componente que conoce contraseñas, roles y la clave privada. STEGO nunca recibe la contraseña ni la clave privada; solo verifica la firma del JWT con la clave pública montada en modo de solo lectura y solicita a AUTH la decisión de autorización para la acción concreta.

Tabla 1: Servicios de Backend V1.

Servicio	Puerto	Responsabilidad
`auth-db`	Interno	PostgreSQL 18 Alpine para identidades, roles, asignaciones y `token_version`.
`auth-migrate`	—	Ejecuta las migraciones de AUTH una sola vez y termina.
AUTH	`3001`	Identidades, roles, asignaciones N:M, JWT RS256, RBAC dinámico y autorización interna con `X-Service-Key`.
`stego-db`	Interno	PostgreSQL 18 Alpine para los resultados de análisis.
`stego-migrate`	—	Ejecuta las migraciones de STEGO una sola vez y termina.
STEGO	`3002`	Validación de imágenes, detector LSB, clasificación y persistencia de resultados.

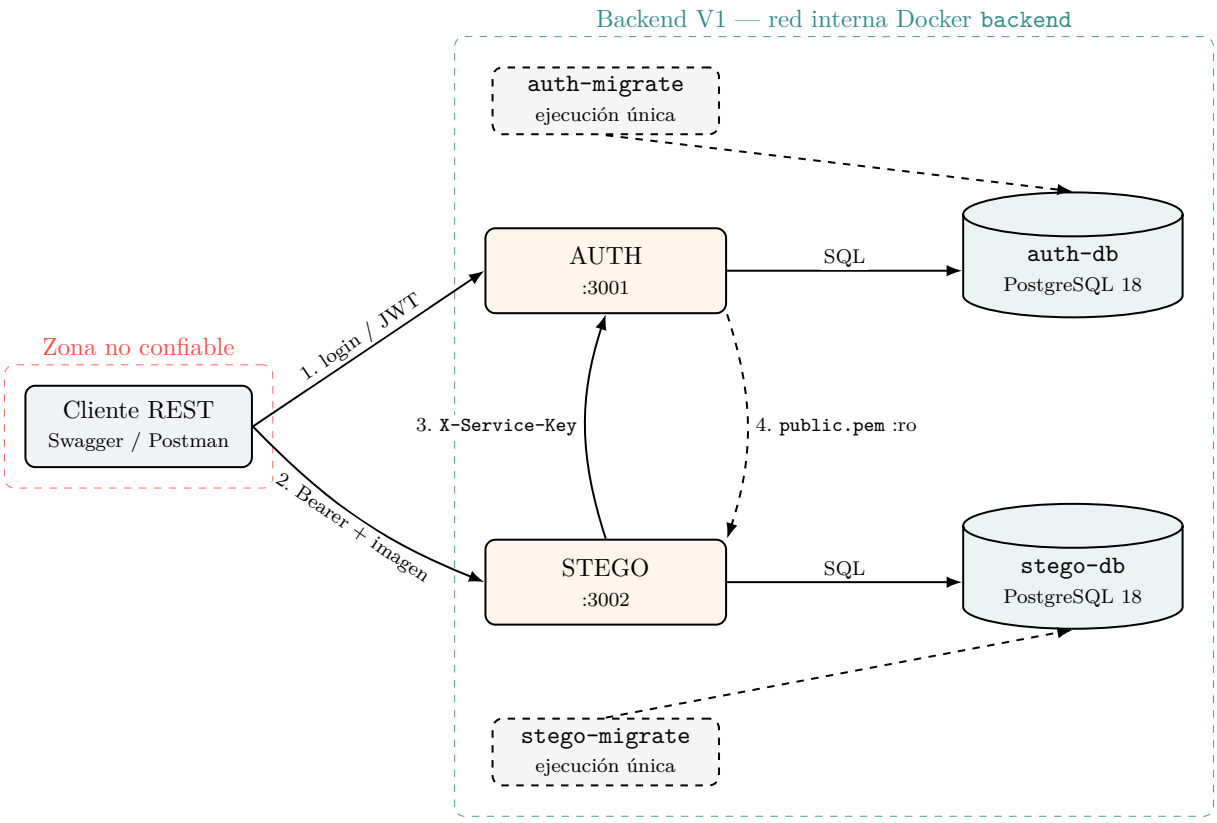


Figura 1: Arquitectura de componentes y límites de confianza. Las bases de datos no publican puertos al host; las APIs se publican solo en `127.0.0.1`.

3.2. Flujo de una solicitud de análisis

1. El usuario se autentica en `POST /api/auth/login`. AUTH verifica la contraseña con Argon2id y emite un JWT RS256 de corta duración; los roles se devuelven en el cuerpo de la respuesta, no dentro del token.
2. El cliente envía el JWT y la imagen a `POST /api/steganography/analyze`.
3. STEGO verifica firma, algoritmo, emisor, audiencia, expiración y tipo `access`; la clave pública se monta desde `public.pem` en modo de solo lectura.
4. STEGO llama a AUTH mediante `POST /internal/authorization/check`, enviando el encabezado `X-Service-Key` y el JWT del usuario en el cuerpo como `subject_token`.
5. AUTH compara la clave de servicio en tiempo constante, verifica el JWT, resuelve el rol efectivo en PostgreSQL y comprueba que la acción esté permitida.
6. Si AUTH no responde en el tiempo configurado (`AUTH_TIMEOUT_MS`, 3000 ms), o la decisión es incoherente, STEGO devuelve `503` y **no procesa el archivo**.
7. Si la autorización es válida, STEGO valida el archivo, ejecuta el detector, persiste el resultado y responde con la clasificación.

3.3. Límites de confianza

Tabla 2: Controles en los límites de confianza.

Límite	**Amenaza principal**	**Control implementado**
Cliente → AUTH	Credenciales débiles o fuerza bruta	Argon2id, validación con DTO y Zod, *rate limit* y mensajes genéricos.
Cliente → STEGO	Archivo malicioso o token robado	Verificación RS256, autorización vigente en AUTH, límites de tamaño y validación por *magic bytes*.
STEGO → AUTH	Suplantación entre servicios	Red interna, `X-Service-Key` de 256 bits, comparación en tiempo constante y cuerpo con `subject_token`.
AUTH → `auth-db`	Acceso directo a datos	Base sin puerto público, consultas parametrizadas por TypeORM y credenciales por variables de entorno.
STEGO → `stego-db`	Lectura de resultados ajenos	Base aislada, sin tablas de AUTH, verificación de propiedad y acción `read.any` en AUTH.
Claves RSA	Fuga de la clave privada	`private.pem` solo en AUTH y en un volumen de solo lectura; STEGO monta únicamente `public.pem`.

Tabla 3: Tecnologías y propósito.

Tecnología	Uso en el proyecto
Node.js 24 (Alpine) y NestJS 12	Runtime y framework de ambos servicios.
TypeORM	Entidades, migraciones versionadas y consultas parametrizadas.
PostgreSQL 18 Alpine	Persistencia relacional separada para AUTH y STEGO.
`@nestjs/jwt` + `passport-jwt`	Firma y verificación JWT RS256; sin algoritmos simétricos.
`argon2`	Hash de contraseñas con Argon2id.
`sharp`	Decodificación de imágenes, normalización a sRGB y análisis LSB.
`multer` + `FileInterceptor`	Recepción en memoria de un único archivo multipart.
`class-validator` + `zod`	Validación de DTO y del entorno de ejecución.
`@nestjs/throttler`	Límite global y límites específicos en login y escrituras.
`@nestjs/swagger`	Contrato OpenAPI y Swagger UI en `/docs`.
Vitest y Supertest	Pruebas unitarias y e2e.
Docker Compose	Orquestación de seis servicios, red, volúmenes y healthchecks.

3.4. Tecnologías

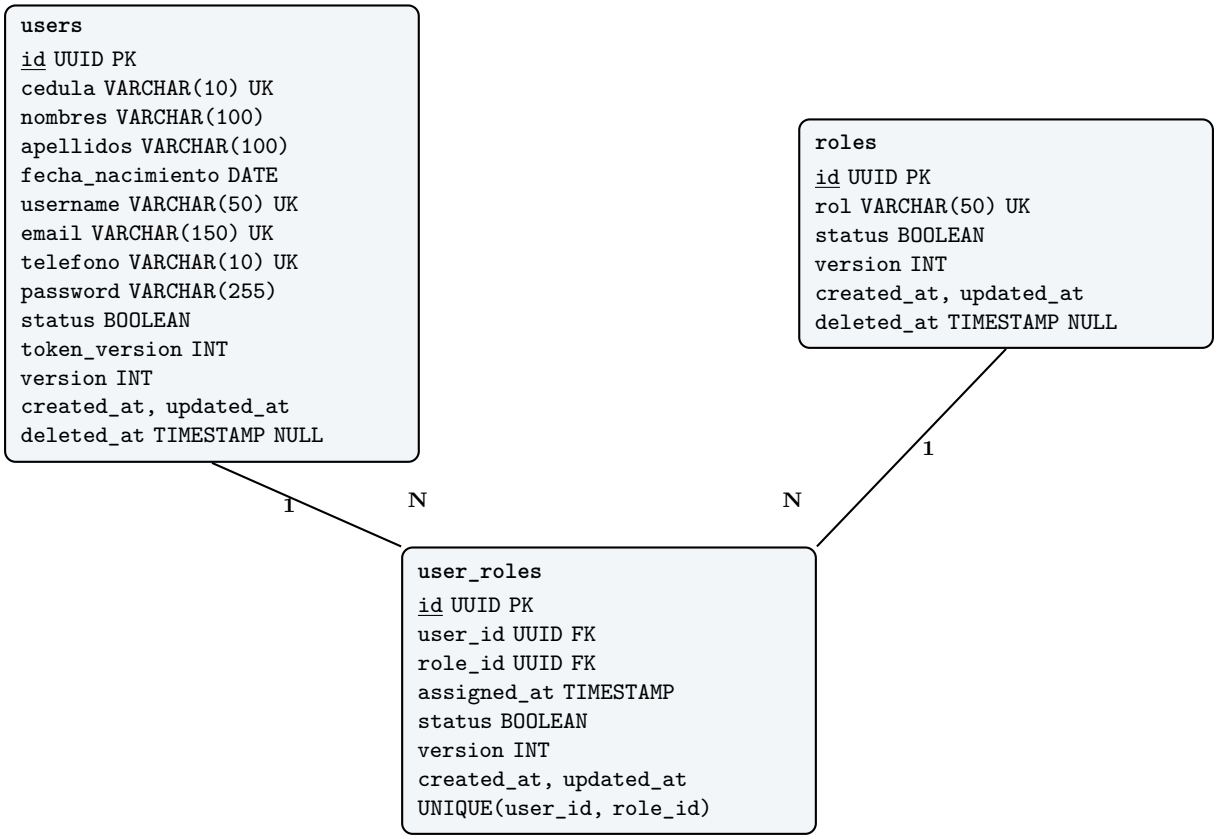
4. Modelo de identidad y datos

4.1. Modelo entidad–relación

El modelo separa identidad, catálogo de roles y asignaciones. La tabla intermedia `user_roles` tiene un índice único `(user_id, role_id)` y claves foráneas con `ON DELETE CASCADE`; esto impide duplicados a nivel de base de datos y elimina asignaciones huérfanas. La revocación de sesiones no usa una tabla de tokens: se implementa con la columna `token_version` en `users`.

4.2. Esquema principal

```
CREATE TABLE IF NOT EXISTS "users" (
  "id" uuid NOT NULL DEFAULT gen_random_uuid(),
  "cedula" varchar(10) NOT NULL,
  "nombres" varchar(100) NOT NULL,
  "apellidos" varchar(100) NOT NULL,
  "fecha_nacimiento" date NOT NULL,
  "username" varchar(50) NOT NULL,
  "email" varchar(150) NOT NULL,
  "telefono" varchar(10) NOT NULL,
  "password" varchar(255) NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "token_version" integer NOT NULL DEFAULT 0,
  "version" integer NOT NULL DEFAULT 1,
  "created_at" TIMESTAMP NOT NULL DEFAULT now(),
  "updated_at" TIMESTAMP NOT NULL DEFAULT now(),
  "deleted_at" TIMESTAMP NULL,
  CONSTRAINT "PK_users" PRIMARY KEY ("id")
);
```



Cada usuario puede tener varios roles y cada rol varios usuarios.

Figura 2: Modelo entidad–relación de AUTH.

```
CREATE TABLE IF NOT EXISTS "roles" (
  "id" uuid NOT NULL DEFAULT gen_random_uuid(),
  "rol" varchar(50) NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "version" integer NOT NULL DEFAULT 1,
  "created_at" TIMESTAMP NOT NULL DEFAULT now(),
  "updated_at" TIMESTAMP NOT NULL DEFAULT now(),
  "deleted_at" TIMESTAMP NULL,
  CONSTRAINT "PK_roles" PRIMARY KEY ("id")
);

CREATE TABLE IF NOT EXISTS "user_roles" (
  "id" uuid NOT NULL DEFAULT gen_random_uuid(),
  "user_id" uuid NOT NULL,
  "role_id" uuid NOT NULL,
  "assigned_at" TIMESTAMP NOT NULL DEFAULT now(),
  "status" boolean NOT NULL DEFAULT true,
  "version" integer NOT NULL DEFAULT 1,
  "created_at" TIMESTAMP NOT NULL DEFAULT now(),
  "updated_at" TIMESTAMP NOT NULL DEFAULT now(),
  CONSTRAINT "PK_user_roles" PRIMARY KEY ("id"),
  CONSTRAINT "FK_user_roles_user" FOREIGN KEY ("user_id")
    REFERENCES "users"("id") ON DELETE CASCADE,
  CONSTRAINT "FK_user_roles_role" FOREIGN KEY ("role_id")
    REFERENCES "roles"("id") ON DELETE CASCADE
);
```

Listing 1: Extracto de `ms-auth-nestjs/src/database/migrations/1700000000000-InitialSchema.ts`.

4.3. Datos semilla

La migración y el *seed* son idempotentes. La contraseña de los usuarios de demostración es `ClaveSegura2026!` únicamente para desarrollo local; en un entorno real debe reemplazarse. El archivo `.env` real está ignorado y no debe distribuirse.

Tabla 4: Usuarios semilla y roles.

Usuario	**Roles**	**Propósito**
`jperez`	`ADMIN`	Administración completa.
`cvega`	`USER`, `ANALYST`	Demuestra múltiples roles simultáneos.
`amendoza`	`ANALYST`	Análisis y consulta de resultados propios.
`mgomez`	`USER`	Sujeto sin permiso de análisis.
`lcastro`	`USER`	Sujeto sin permiso de análisis.

4.4. Persistencia de STEGO

STEGO usa su propia base de datos PostgreSQL, separada de AUTH. Guarda cada análisis en la tabla `stego_analysis` con los campos `id`, `user_id`, `filename`, `sha256`, `technique`, `status`, `indicators` (JSONB), `action`, `limitations` y `created_at`. El SHA-256 se calcula sobre los bytes originales recibidos, antes de cualquier decodificación. La imagen no se escribe en disco: se procesa en memoria y se descarta al terminar la solicitud. La consulta `GET` verifica que el sujeto sea propietario del resultado o tenga permiso `read.any` mediante AUTH.

5. Microservicio AUTH

5.1. Contraseñas e identidad

Las contraseñas se almacenan exclusivamente como hash **Argon2id** con `memoryCost: 19456` KiB, `timeCost: 2` y `parallelism: 1`. El login crea un hash ficticio al iniciar el módulo y compara siempre contra un hash válido, incluso cuando el usuario no existe, para reducir diferencias de tiempo. Las respuestas de error no revelan si el usuario existe: toda falla de credenciales responde con `Credenciales inválidas`. El campo `password` está excluido de la serialización y no se selecciona por defecto en las consultas.

5.2. JWT RS256

AUTH y STEGO comparten un par RSA de 2048 bits generado fuera de la imagen. La clave privada reside en `ms-auth-nestjs/keys/private.pem` y solo se monta en AUTH; la clave pública se monta en STEGO con la opción `:ro`. No existe endpoint JWKS: la distribución de la clave pública es por volumen de solo lectura. El token se firma con `RS256` y se verifica con algoritmo permitido explícito, emisor, audiencia, expiración y tipo `access`; no se acepta `none` ni algoritmos simétricos.

Tabla 5: Claims del JWT.

Claim	Tipo	Uso
`sub`	UUID	Identificador del usuario.
`username`	string	Nombre de usuario para trazabilidad.
`tokenVersion`	numérico	Versión de sesión para revocación inmediata.
`typ`	string	Debe ser `access`; otros valores se rechazan.
`iss`	string	Emisor esperado: `ms-auth`.
`aud`	string	Audiencia esperada: `secure-chat-api`.
`iat`, `exp`	numérico	Emisión y expiración (900 segundos por defecto).

Los roles **no** se incluyen en el JWT. El login devuelve `accessToken`, `tokenType` y un objeto `user` con el nombre de usuario y los roles vigentes leídos de PostgreSQL.

5.3. RBAC vigente

Los roles se consultan en la base de datos en cada solicitud, por lo que un cambio de rol surte efecto de inmediato aunque el token siga vigente criptográficamente. Si a un usuario se le retira un rol, su token anterior deja de servir para esa acción. Lo mismo ocurre con asignaciones desactivadas: el *guard* de roles deniega por defecto toda ruta privada que no declare roles o la marca de acceso autenticado.

Tabla 6: Matriz de roles y permisos.

Acción	ADMIN	ANALYST	USER
Gestionar usuarios, roles y asignaciones	Sí	No	No
`steganography.analyze`	Sí	Sí	No
`steganography.results.read.own`	Sí	Sí	Sí
`steganography.results.read.any`	Sí	No	No
Consultar perfil propio (`/api/auth/profile`)	Sí	Sí	Sí

5.4. Autorización interna entre servicios

STEGO no puede confiar únicamente en el JWT para decidir una acción, porque el usuario pudo perder el rol o cerrar sesión después de la emisión. Por eso consulta a AUTH. La solicitud interna incluye el encabezado `X-Service-Key`, cuyo valor es una clave compartida de 256 bits (`INTERNAL_API_KEY`) comparada en tiempo constante, y un cuerpo con `subject_token`, `action` y `resource_id` opcional. AUTH verifica el JWT, carga el usuario y sus roles activos desde PostgreSQL, comprueba `tokenVersion` y responde con una decisión explícita.

```
POST /internal/authorization/check
X-Service-Key: <INTERNAL_API_KEY>

{
  "subject_token": "<JWT del usuario>",
  "action": "steganography.analyze",
  "resource_id": null
}
```

Listing 2: Cuerpo de la consulta interna de autorización.

AUTH distingue los motivos `subject_not_active`, `token_revoked`, `unsupported_action` e `insufficient_permissions`. Si falla la persistencia después de validar el token, responde `allowed: false` con motivo `authorization_unavailable`. STEGO aplica *fail closed*: si AUTH no responde, devuelve `503` y no procesa ni persiste el archivo.

5.5. Revocación y cierre de sesión

`POST /api/auth/logout` incrementa la columna `users.token_version`. El JWT incluye el valor de `tokenVersion` al momento de la emisión; en cada verificación se compara con el valor actual de la base. Una diferencia produce `401` con el mensaje `La sesión ha expirado`. Esta

revocación sobrevive a reinicios del servicio porque reside en PostgreSQL y no en memoria. No existe una tabla de tokens revocados ni un mecanismo de refresh token.

5.6. Endpoints de AUTH

Tabla 7: Endpoints de AUTH.

Método	Ruta	Acceso	Respuesta
POST	`/api/auth/login`	Público, rate limit 5/min	`200` token / `401`
POST	`/api/auth/logout`	Autenticado	`204`
GET	`/api/auth/profile`	Autenticado	`200` perfil
GET	`/api/auth/me`	Autenticado (alias)	`200` perfil
GET	`/api/users`	ADMIN	`200` lista paginada
POST	`/api/users`	ADMIN	`201` / `409`
GET	`/api/users/cedula/:cedula`	ADMIN	`200` / `404`
GET	`/api/users/:id`	ADMIN	`200` / `404`
PATCH	`/api/users/:id`	ADMIN	`200`
DELETE	`/api/users/:id`	ADMIN	`204` (borrado lógico)
GET	`/api/roles`	ADMIN	`200`
POST	`/api/roles`	ADMIN	`201` / `409`
GET	`/api/roles/:id`	ADMIN	`200` / `404`
PATCH	`/api/roles/:id`	ADMIN	`200`
DELETE	`/api/roles/:id`	ADMIN	`204` (borrado lógico)
GET	`/api/user-roles/user/:userId`	ADMIN	`200`
POST	`/api/user-roles`	ADMIN	`201` / `409` / `404`
GET	`/api/user-roles/:id`	ADMIN	`200` / `404`
PATCH	`/api/user-roles/:id`	ADMIN	`200`
DELETE	`/api/user-roles/:id`	ADMIN	`204`
DELETE	`/api/user-roles/:userId/:roleId`	ADMIN	`204`
POST	`/internal/authorization/check`	`X-Service-Key`	`200` decisión / `401`
GET	`/api/health`	Público	`200`
GET	`/docs`	Público local	Swagger UI

5.7. Controles adicionales

- Consultas SQL parametrizadas mediante TypeORM; no se concatena entrada del usuario.
- `ValidationPipe` con `whitelist`, `forbidNonWhitelisted` y `forbidUnknownValues`; errores sin valores de entrada.
- *Rate limit* global y límites específicos: login 5/min, creación de usuarios 10/min, roles y asignaciones 20/min, autorización interna 60/min.

- Errores uniformes sin pilas de excepción; el filtro global registra método, URL y traza solo para errores 5xx.
- CORS restringido por lista de orígenes; `X-Service-Key` no se expone a navegadores.
- Cabeceras de seguridad provistas por NestJS, `X-Request-Id` validado y contenedor ejecutado como usuario no privilegiado.

6. Microservicio STEGO

6.1. Pipeline de validación

Antes de leer píxeles, STEGO aplica una secuencia de rechazo temprano. Esto evita consumo de memoria y CPU con archivos que no cumplen el contrato. El `Content-Type` declarado por el cliente se ignora deliberadamente: la decisión se basa en la extensión y en los *magic bytes* reales.

Tabla 8: Validaciones de entrada de STEGO.

Control	**Regla**	**Error**
Autenticación	JWT RS256 válido, tipo `access`, no expirado	`401`
Autorización	`steganography.analyze` vigente en AUTH	`403`
Disponibilidad de AUTH	Respuesta coherente dentro de 3000 ms	`503`
Campo de archivo	Exactamente un archivo en `file`	`400`
Tamaño	Máximo 10 MiB (`MAX_FILE_SIZE_BYTES`)	`413`
Extensión	`.png` o `.bmp`	`415`
Firma binaria	PNG `89 50 4E 47 0D 0A 1A 0A` o BMP `42 4D`	`415`
Dimensiones	Máximo 10000 por lado y 40 megapíxeles	`413`
Decodificación	`sharp` normaliza a sRGB sin alfa; exige 3 canales	`422`

La recepción usa `multer` con almacenamiento en memoria y el límite de tamaño como primera barrera. El nombre original se sanea eliminando componentes de ruta y caracteres de control. Los errores de formato no exponen detalles internos del decodificador. El archivo original no se ejecuta, no se sirve públicamente y no se persiste en disco.

6.2. Detección de esteganografía LSB

La técnica LSB reemplaza los bits menos significativos de los canales de color. Una imagen natural suele tener alta variabilidad en esos bits; una imagen manipulada tiende a igualar la frecuencia de valores pares e impares y a aumentar la uniformidad. El detector combina tres señales calculadas sobre los canales RGB agregados y también por canal individual.

Entropía LSB. Se calcula la entropía de Shannon de los bits menos significativos:

$$H = -\sum_{i=0}^{1} p_i \log_2 p_i. \tag{1}$$

Una imagen natural no suele acercarse a $H = 1$; una imagen con carga aleatoria se aproxima a ese valor. El indicador se acota al intervalo $[0, 1]$.

Igualación de pares (PoV). Para cada par de valores $(2i, 2i+1)$ se cuentan las apariciones de cada uno y se acumula la diferencia absoluta. La puntuación es

$$P = 1 - \frac{\sum_{i=0}^{127} |n_{2i} - n_{2i+1}|}{N}, \tag{2}$$

donde N es el total de bytes analizados. Un valor alto indica pares equilibrados, compatible con reemplazo LSB.

Chi-cuadrado con Wilson-Hilferty. Para cada par no vacío se calcula el estadístico

$$\chi^2 = \sum_{i=0}^{127} \left(\frac{(n_{2i} - E_i)^2}{E_i} + \frac{(n_{2i+1} - E_i)^2}{E_i} \right), \qquad E_i = \frac{n_{2i} + n_{2i+1}}{2}. \tag{3}$$

El p-valor se aproxima con la transformación de Wilson-Hilferty y la función error de Abramowitz y Stegun. Es una aproximación documentada, no un cálculo exacto de la función gamma incompleta.

Puntaje final. La versión implementada usa:

$$\text{anomalyScore} = 0{,}45\, H + 0{,}45\, P + 0{,}10\, p_{\chi^2}. \tag{4}$$

Los pesos priorizan la entropía y la igualación de pares, y usan el p-valor de chi-cuadrado como señal de apoyo. La calibración se realizó con imágenes sintéticas controladas y debe revalidarse con un corpus etiquetado más amplio antes de producción.

6.3. Clasificación y política

Tabla 9: Política de clasificación.

Estado	Condición	Interpretación	Acción
`sin_indicios`	score $\leq 0{,}55$	No se observan señales fuertes de LSB.	`aceptado`
`no_concluyente`	$0{,}55 <$ score $< 0{,}75$	Señales insuficientes para decidir.	`advertencia`
`sospechoso`	score $\geq 0{,}75$	Evidencia compatible con carga LSB.	`cuarentena`
`error`	Decodificación fallida	La imagen superó la validación pero no pudo decodificarse.	`rechazo`

Los umbrales son configurables:

- `STEGO_CLEAR_THRESHOLD`: 0.55.
- `STEGO_SUSPICIOUS_THRESHOLD`: 0.75.

El esquema de entorno exige que el umbral claro sea menor que el sospechoso. La política no elimina ni ejecuta el archivo. Si la clasificación es sospechosa, se marca la acción `cuarentena` y el resultado queda disponible para revisión manual junto con los indicadores estadísticos.

6.4. Resultados, aislamiento y trazabilidad

Cada análisis se persiste antes de responder, de modo que un fallo posterior no deja una decisión sin registro. El identificador se genera con `randomUUID`. La consulta de resultados exige autenticación y aplica dos controles: existencia del recurso y autorización en AUTH. Para el propietario se exige `steganography.results.read.own`; para un resultado ajeno se exige `steganography.results.read.any`. Un `USER` puede leer sus propios resultados, pero no los ajenos; un `ANALYST` tampoco puede leer los ajenos; un `ADMIN` sí.

Los servicios escriben eventos de acceso HTTP, autenticación, decisión de autorización y análisis. El filtro global no registra cuerpos de solicitud, y el campo de contraseña está excluido de la serialización. No se registran contraseñas, JWT completos ni la clave de servicio.

6.5. Endpoints de STEGO

Tabla 10: Endpoints de STEGO.

Método	**Ruta**	**Acceso**	**Respuesta**
POST	`/api/steganography/analyze`	ADMIN, ANALYST	`201` / `401` / `403` / `413` / `415` / `422` / `503`
GET	`/api/steganography/results/:id`	Propietario o ADMIN	`200` / `403` / `404` / `503`
GET	`/api/health`	Público	`200` / `503`
GET	`/docs`	Público local	Swagger UI

6.6. Ejemplo de resultado

```
{
  "id": "0f8c...",
  "userId": "uuid-del-analista",
  "filename": "Logo.png",
  "sha256": "9f2c...",
  "technique": "lsb_statistical_analysis",
  "status": "sospechoso",
  "indicators": {
    "lsbEntropy": 0.98,
    "poVScore": 0.97,
    "chiSquarePValue": 0.42,
    "anomalyScore": 0.92,
    "threshold": 0.75,
    "width": 512,
    "height": 512,
    "perChannel": { "red": {}, "green": {}, "blue": {} }
  },
  "action": "cuarentena",
  "limitations": "Analisis estadistico de triage; no demuestra ausencia de
    informacion oculta.",
  "createdAt": "2026-11-04T00:00:00.000Z"
}
```

Listing 3: Respuesta resumida de un análisis. Los valores numéricos dependen de la imagen.

7. Seguridad, riesgos y controles

7.1. Enfoque de seguridad

El diseño adopta tres ideas rectoras. *Zero Trust* implica no confiar en el cliente ni en el contenido del token para decisiones sensibles: cada solicitud se verifica y la autorización se consulta a la fuente de verdad. *Secure by Design* implica que los límites, validaciones y permisos se definen antes de programar el flujo feliz. *Shift Left* implica probar los abusos —token manipulado, rol retirado, sesión revocada, AUTH caído, archivo corrupto, resultado ajeno— en la misma suite que las operaciones normales.

Estos principios se alinean con ISO/IEC 27001 (gestión de seguridad), ISO/IEC 27034 (seguridad de aplicaciones), OWASP ASVS y las recomendaciones de RFC 8725 para JWT. El proyecto no declara certificación; usa las normas como marco de referencia.

Tabla 11: Matriz de activos, amenazas y controles esperados.

Activo	**Amenaza**	**Control Esperado e Implementado**
Credenciales e identidades (`users`)	Ataques de fuerza bruta, robo o fuga de contraseñas, enumeración de cuentas e inyección de datos maliciosos en DTOs.	Derivación de clave mediante Argon2id con salt criptográfico aleatorio, validación estricta en DTOs con Zod en `ValidationPipe`, exclusión de hashes con `ClassSerializerInterceptor`, limitador de tasa (`ThrottlerModule`) y respuestas uniformes y genéricas ante fallos de login.
Matriz RBAC y asignaciones N:M (`user_roles`)	Elevación de privilegios, persistencia indebida de accesos tras revocación de rol y duplicación de asignaciones.	Integridad referencial con restricción `UNIQUE(user_id, role_id)`, consulta dinámica en PostgreSQL en tiempo real en cada operación sensible (sin confiar en claims estáticos de tokens), endpoints de gestión restringidos a ADMIN, borrado lógico (`deletedAt`) y revocación inmediata en caliente.
Claves criptográficas RSA (RS256)	Exfiltración de la clave privada, manipulación no autorizada de firmas digitales o exposición de secretos en imágenes Docker.	Asimetría estricta de responsabilidades: `private.pem` montada en modo solo lectura (`:ro`) exclusivamente en AUTH fuera de la imagen; STEGO solo monta `public.pem:ro`; eliminación definitiva de secretos compartidos simétricos (`JWT_SECRET`).
Tokens de acceso JWT (Sesión)	Falsificación o manipulación de tokens, ataques de degradación de algoritmo (*none algorithm* o HS256) y reutilización post-expiración.	Firma digital asimétrica RS256 (2048 bits), tiempo de vida corto (15 min / 900 s), validación rigurosa de claims (`iss=ms-auth`, `aud=secure-chat-api`, `sub`, `typ=access`) y rechazo estricto de algoritmos no autorizados en `JwtStrategy`.
Canal inter-servicio y autorización interna	Suplantación de peticiones de STEGO, ataques de temporización (*timing attacks*), inyección de tráfico externo o autorización por defecto ante caída.	Aislamiento en red interna Docker tipo bridge (`backend`), autenticación mutua con cabecera `X-Service-Key` de 256 bits (64 hex), verificación en tiempo constante con `crypto.timingSafeEqual`, throttler (60 req/min) y política rigurosa *Fail-Closed* (HTTP 503).
Carga y análisis de imágenes (PNG/BMP)	Inyección de malware, bombas de descompresión (*pixel bombs* y DoS por saturación de RAM), evasión por tipo MIME y corrupción de memoria.	Validación en memoria de *magic bytes* (`89 50 4E 47` y `42 4D`), límite de tamaño de archivo a 10 MiB, restricción dimensional (máx. 10.000×10.000 px, 40 MP), decodificación aislada en memoria volátil con `sharp` (sin escribir en disco) y cálculo previo de hash SHA-256.

Activo	Amenaza	Control Esperado e Implementado
Historial de análisis y evidencia (`stego_analysis`)	Acceso no autorizado a análisis ajenos, manipulación o adulteración de hallazgos forenses y predicción de identificadores de registro.	Identificadores opacos UUID v4 generados en backend con `randomUUID()`, inmutabilidad del hash SHA-256 e indicadores estadísticos (Entropía, PoV, χ^2), y validación de propiedad en AUTH (`read.own` vs `read.any`).
Motores de base de datos (PostgreSQL)	Acceso directo no autorizado a puertos de datos desde redes públicas o el host, inyección SQL o corrupción de volúmenes persistentes.	Instancias PostgreSQL independientes (`auth-db` y `stego-db`) completamente aisladas en la red interna Docker (sin puertos expuestos al host), consultas parametrizadas con TypeORM, servicios de migración transaccionales e idempotentes (`auth-migrate`, `stego-migrate`) y volúmenes externos dedicados.
Superficie de API y Swagger / OpenAPI	Enumeración de endpoints, acceso no autorizado a documentación OpenAPI en entornos productivos o fugas de trazas internas en excepciones.	Documentación Swagger (`/docs`) publicada exclusivamente en la interfaz de bucle local (`127.0.0.1`), condicionada dinámicamente por la variable `SWAGGER_ENABLED`, exclusión de `/internal/authorization/check` del prefijo `/api`, cabeceras HTTP de endurecimiento con `helmet`, CORS restrictivo y filtro global `AllExceptionsFilter`.

7.2. Decisiones de diseño relevantes

- **El JWT no es la autorización final.** STEGO consulta AUTH para respetar cambios de rol y revocaciones inmediatas.
- **La clave privada nunca sale de AUTH.** STEGO solo monta `public.pem` en modo de solo lectura.
- **El archivo no se sirve ni se ejecuta.** Se decodifica en memoria y solo se persiste el resultado y su SHA-256.
- **Los errores no filtran detalles.** El cliente recibe códigos y mensajes estables; el detalle técnico queda en logs controlados.
- **Las bases de datos están aisladas.** Cada servicio tiene su propio PostgreSQL, su propio volumen externo y su propio ciclo de migraciones.
- **La evidencia es reproducible.** Las pruebas unitarias generan imágenes sintéticas controladas y no dependen de archivos externos.

8. Pruebas y resultados

8.1. Estrategia

La verificación combina pruebas unitarias y pruebas de extremo a extremo. Las unitarias cubren funciones puras de seguridad, detección, validación y coordinación con AUTH, sin base de datos. Las e2e de AUTH usan HTTP real, PostgreSQL real, migraciones reales y tokens firmados con RSA. STEGO no cuenta actualmente con un arnés e2e propio; su verificación se apoya en pruebas unitarias que cubren el detector, la validación de archivos, el cliente de autorización y la lógica de persistencia.

8.2. Pruebas unitarias

Tabla 12: Resultados de pruebas unitarias ejecutadas.

Servicio	Archivos	Resultado	Cobertura funcional
AUTH	6	36/36	Health, Argon2id, emisión de JWT, guard de roles, autorización interna completa y validación de `X-Service-Key`.
STEGO	10	47/47	Cliente de AUTH, estrategia JWT, esquema de entorno, migración, health, servicio de análisis, entidad, validación de archivos, métricas LSB y persistencia.
Total	**16**	**83/83**	Ejecutado con Vitest 4.1.12.

Evidencia de ejecución

La adaptación se validó con `npm test` en ambos servicios el 04 de noviembre de 2026: AUTH reportó **6 archivos y 36 pruebas aprobadas**; STEGO reportó **10 archivos y 47 pruebas aprobadas**. No se registraron fallos.

8.3. Pruebas de extremo a extremo de AUTH

La suite `ms-auth-nestjs/test/ms-auth.e2e-spec.ts` contiene **30 pruebas** contra una instancia real de PostgreSQL. Verifica el healthcheck, la ausencia de rutas `/api/v1`, la aplicación de migraciones, el login con credenciales válidas e inválidas, el perfil, el logout con revocación por `tokenVersion` y el endpoint interno con claves de servicio y tokens manipulados.

8.4. Cobertura de los escenarios T01–T12

La Tabla 13 relaciona los escenarios solicitados por la guía con la evidencia disponible en el repositorio. Se distingue entre cobertura e2e, cobertura unitaria y verificación manual, para no atribuir a la suite automatizada pruebas que no existen.

Tabla 13: Cobertura de los escenarios T01–T12.

ID	Escenario	Evidencia real	Estado
T01	Arranque autónomo y servicios separados	Compose con seis servicios; healthchecks en `/api/health`; e2e de health de AUTH; unitaria de health de STEGO	Cubierto
T02	Credenciales válidas e inválidas	e2e de AUTH: login exitoso, usuario inexistente, contraseña incorrecta y usuario inactivo	Cubierto (e2e)
T03	Token ausente o manipulado	e2e de AUTH: sin token, HS256, otra clave RSA, expirado, emisor y audiencia incorrectos, `typ` distinto; unitaria de estrategia JWT de STEGO	Cubierto
T04	Rechazo 403 por rol	e2e interno: USER denegado para análisis; unitarias de guard de roles y de *fail closed*	Cubierto
T05	CRUD y asignaciones N:M	Endpoints implementados; unitarias de servicio y guard; sin e2e dedicado de CRUD	Parcial
T06	Duplicidad N:M	Restricción `UNIQUE(user_id, role_id)` y respuesta `409` en el servicio; sin e2e dedicado	Parcial
T07	Revocación en caliente por `tokenVersion`	e2e: logout invalida el token anterior; decisión interna deniega token revocado	Cubierto (e2e)
T08	Fail-Closed 503 por AUTH no disponible	Unitarias de `AuthClientService`: timeout, error de red, 5xx, JSON inválido y decisión incoherente producen `503`; no se procesa el archivo	Cubierto (unitaria)
T09	Análisis LSB real	Unitarias con imágenes sintéticas: host vs LSB incrustado, reproducibilidad, métricas agregadas y por canal	Cubierto (unitaria)
T10	Magic bytes, 413 y 415	Unitarias de validación: PNG, BMP, `Content-Type` ignorado, extensión/firma incoherente, formato no soportado y tamaño excesivo	Cubierto (unitaria)
T11	Propiedad del resultado	e2e interno de `read.own/read.any`; unitarias de STEGO para propietario, ajeno, 404 sin consultar AUTH y ocultamiento de la denegación	Cubierto
T12	Registros depurados	Revisión manual de logs y del filtro global; no existe prueba automatizada de redacción	Manual

8.5. Resultados sobre imágenes sintéticas

Las comparaciones son relativas a imágenes sintéticas controladas y no constituyen una afirmación de precisión forense general. La ventaja frente a una suma simple de LSB es el uso combinado de señales; la limitación principal sigue siendo la esteganografía adaptativa.

Tabla 14: Fixtures sintéticos y comportamiento verificado.

Fixture	**Condición**	**Resultado verificado**
`createHostPng(48,48)`	Imagen controlada sin carga	Entropía LSB y PoV cercanos a cero; `anomalyScore` menor que el de la variante con LSB.
`createLsbEmbeddedPng(48,48)`	Bits LSB inyectados	Entropía y PoV mayores que el host; `anomalyScore` supera el umbral sospechoso.
`createLsbEmbeddedPng(16,16)`	Métricas por canal	768 bytes analizados; métricas RGB no nulas y p-valor acotado a $[0, 1]$.
PNG truncado	Firma PNG y relleno inválido	`ImageDecodingError`; el servicio persiste estado `error` y responde 422.

8.6. Reproducibilidad

```
# Desde la raiz del proyecto
docker volume create ms-auth-nestjs_users_postgres_data
docker volume create ms-stego-nestjs_stego_postgres_data
docker compose config --quiet
docker compose up -d --build

# Estado y documentacion
docker compose ps -a
# AUTH:  http://localhost:3001/docs
# STEGO: http://localhost:3002/docs

# Datos semilla
docker compose exec auth node dist/database/seed.js

# Pruebas unitarias
cd ms-auth-nestjs;  npm ci; npm test; npm run lint; npm run build
cd ../ms-stego-nestjs; npm ci; npm test; npm run lint; npm run build

# Pruebas e2e de AUTH (requiere PostgreSQL real)
cd ../ms-auth-nestjs; npm run test:e2e

# Apagado sin eliminar volumenes externos
cd ..; docker compose down
```

Listing 4: Comandos de operación y verificación en PowerShell.

Las migraciones se ejecutan en los servicios `auth-migrate` y `stego-migrate`, que terminan antes de que arranquen las APIs. Los healthchecks evitan que STEGO arranque antes de que AUTH esté saludable. Los volúmenes externos conservan los datos entre reinicios y no deben eliminarse con `docker compose down -v`.

9. Anexos

9.1. Link del Repositorio

https://github.com/JersonL1M23/secure-chat-proyecto-DJF.git

9.2. Funcionalidad del aplicativo



Figura 3: Acceso válido al sistema usando las credenciales de ADMIN

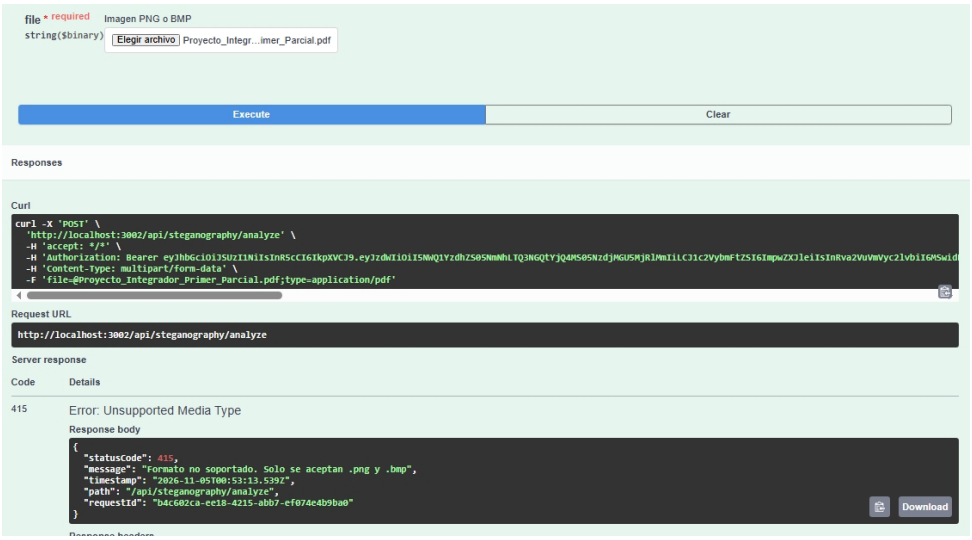


Figura 4: Validación de esteganografía de un archivo PDF (Rechaza el archivo)

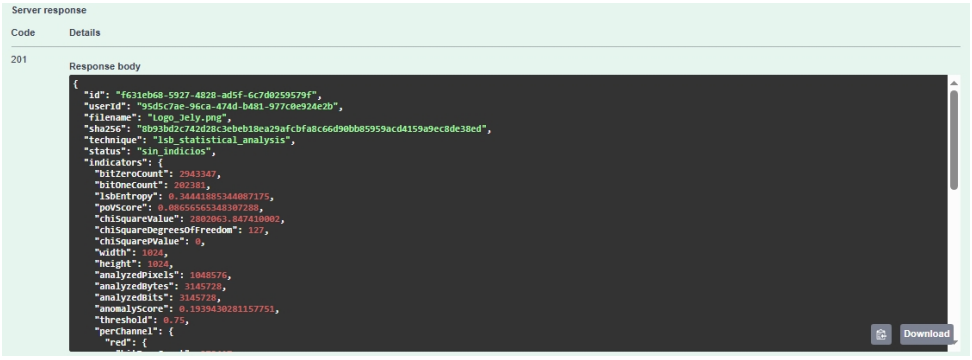


Figura 5: Aceptación de un archivo permitido (png)

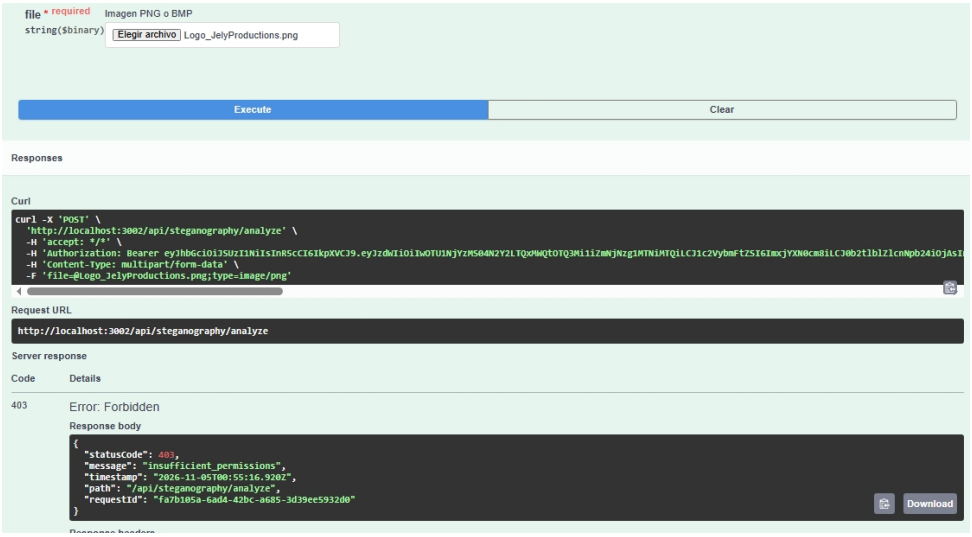


Figura 6: Intento de analizar esteganografía desde rol usuario (Insuficientes permisos)

9.3. Estructura del entregable

```
secure-chat-proyecto-DJF/
  docker-compose.yml       Orquestacion de auth-db, auth-migrate, auth,
                           stego-db, stego-migrate y stego
  .env.example             Plantilla de variables de entorno
  README.md                Guia de operacion y pruebas
  ms-auth-nestjs/          Servicio AUTH (NestJS), migraciones, seed,
                           pruebas unitarias y e2e, claves locales
  ms-stego-nestjs/         Servicio STEGO (NestJS), detector LSB,
                           validacion, migraciones y pruebas unitarias
  Informe/                 Este informe en LaTeX y PDF
```

Listing 5: Organización del proyecto.

9.4. Resumen de contratos OpenAPI

AUTH expone su contrato Swagger en `http://localhost:3001/docs` y STEGO en `http://localhost:3002/docs` cuando `SWAGGER_ENABLED=true`. Los contratos documentan esquemas de entrada, respuestas, códigos de error y seguridad Bearer. Las rutas de negocio usan el prefijo `/api`; el endpoint interno de autorización queda fuera de ese prefijo y se autentica con `X-Service-Key`.

Trazabilidad

Los archivos de código, migraciones, pruebas y contratos están dentro del entregable y pueden regenerarse con los comandos del capítulo de pruebas. No se incluyen secretos reales en la documentación ni en el informe: `.env` y los archivos `*.pem` están ignorados por Git.

10. Conclusiones

La separación entre AUTH y STEGO permitió construir un backend con límites de confianza explícitos. AUTH concentra identidad, claves y decisiones de autorización; STEGO concentra la validación y el análisis de archivos. La consulta de autorización en cada operación sensible resolvió el problema de tokens que sobreviven a un cambio de rol, y la clave de servicio protegió el canal interno sin compartir la clave privada RSA.

El detector LSB cumple el objetivo de triage: distingue imágenes sintéticas de control de variantes con carga LSB, mantiene la reproducibilidad de las métricas y rechaza archivos inválidos sin procesarlos. La combinación de entropía, igualación de pares y chi-cuadrado es más robusta que una sola señal, aunque no sustituye una revisión forense. La principal limitación es la tasa de falsos negativos ante esteganografía adaptativa; el siguiente paso sería ampliar el corpus, calibrar los umbrales con datos etiquetados y agregar análisis de dominio de transformada.

La evidencia obtenida —83 pruebas unitarias aprobadas y una suite e2e de AUTH de 30 pruebas— permite afirmar que la versión entregada es reproducible, operable y coherente con los controles documentados. El proyecto deja además una base clara para incorporar HTTPS, rotación de claves, observabilidad centralizada y despliegue en un entorno con secretos administrados.

Referencias

[1] ISO/IEC. *ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. Ginebra: ISO, 2022.

[2] ISO/IEC. *ISO/IEC 27034-1:2011 — Application security — Part 1: Overview and concepts*. Ginebra: ISO, 2011.

[3] OWASP Foundation. *OWASP Application Security Verification Standard (ASVS)*. Versión 4.0.3, 2021. Disponible en https://owasp.org/www-project-application-security-verification-standard/.

[4] OWASP Foundation. *File Upload Cheat Sheet*. Disponible en https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html.

[5] M. Jones, J. Bradley y N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. IETF, 2015.

[6] M. Jones, J. Bradley y N. Sakimura. *RFC 8725: JSON Web Token Best Current Practices*. IETF, 2020.

[7] S. Rose, O. Borchert, S. Mitchell y S. Connelly. *NIST SP 800-207: Zero Trust Architecture*. NIST, 2020.

[8] C. E. Shannon. *A Mathematical Theory of Communication*. Bell System Technical Journal, 27(3):379–423, 1948.

[9] J. Fridrich, M. Goljan y R. Du. *Reliable Detection of LSB Steganography in Color and Grayscale Images*. Proceedings of the ACM Workshop on Multimedia and Security, 2001.